

作业 8

——MinHash 近似 Jaccard Similarity——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

给出 n 个单重集 $A_1, A_2, \dots, A_n$ ，每个集合都包含 m 个 $[0, 10^9]$ 中的整数。

有 q 次询问，每次询问给出 $i, j (1 \leq i, j \leq n)$ ，你的程序需要给出一个 $|A_i \cap A_j| / |A_i \cup A_j|$ 的近似解。

输入

第一行三个整数 $n, m, q (1 \leq n, m \leq 1000, 1 \leq q \leq 5 \times 10^5)$ 。接下来的 n 行，每行 m 个整数，表示集合 A_i 中的元素。接下来的 q 行，每行两个整数 i, j ，表示一组询问。

输出

对于每组询问，你需要输出一个浮点数，表示你给出的近似解。你需要保证，对于至少 99% 的询问：令你输出的解为 x ，标准答案为 y 。首先有 $x \geq 0$ ；其次要么有 $|x - y| < 0.15$ ，要么有 $\frac{\max(x, y)}{\min(x, y)} < 1.5$ 。

样例输入

```
3 2 2
1 2
2 3
1 3
1 2
1 3
```

样例输出

```
0.3333
0.3333
```

2. 分析

这个题目的意思非常简单：对于两个集合，我们定义他们的相似度是一个 $0, 1$ 之间的浮点数，表示它们交集与并集之比。我们需要给出这个比例的近似值。我们对精度的要求是，要么误差小于 0.15 ，要么相似度的比例不超过 1.5 倍。第二个限制条件是避免当真实值比较小 (< 0.3) 的时候，你随机输出都能正确。

算法思路是 MinHash 算法。具体来说操作步骤是

- 1) 找到很多个哈希函数，并将数据通过哈希函数映射到一个较小的空间中。
- 2) 我们找一个集合映射后最小的元素。
- 3) 对于相同的哈希函数，相同的元素经过映射后也应该得到相同的结果。因此，我们选择比较哈希函数中最小的数值是否相等。如果相等，那么我们就在原象中找到了两个相等的元素。
- 4) 通过计数，在足够多的不同的哈希函数中，统计最小值相等的次数占总哈希函数数量的比例，这个比例就是近似的 Jaccard 相似度。

于是，我们的思路已经很清楚了。接下来我们就来实现这个算法。

3. 算法实现

3.1. 我们需要很多个哈希函数

```
int hashFunction(int x, int t, vector<int>& k, vector<int>& mod){
    return (x ^ k[t]) * 1LL * k[t] % mod[t];
}
```

其中， k , mod 是预先生成的随机数。我们需要保证这些哈希函数的独立性，因此我们需要生成足够多的随机数。需要注意的是，这里的随机数不能是 0，否则除法会不合法。

```
srand(time(0));
// 生成T个随机哈希函数的参数
vector<int> k(T), mod(T);
for(int i = 0; i < T; i++){
    k[i] = rand() % 1000000000 + 1;
    mod[i] = rand() % 1000000000 + 1;
}
```

3.2. 对每个集合计算它的 MinHash

我们需要一个二维数组来存储每个集合在每个哈希函数下的最小哈希值。对于每个集合，我们遍历它的元素，计算它们在每个哈希函数下的哈希值，并更新最小值。

```
vector<vector<int>> minn(n, vector<int>(T, INT_MAX));
for(int i = 0; i < n; i++){
    for(int t = 0; t < T; t++){
        for(int j = 0; j < m; j++){
            int hashValue = hashFunction(a[i][j], t, k, mod);
            minn[i][t] = min(minn[i][t], hashValue);
        }
    }
}
```

3.3. 处理查询

对于每个查询，我们需要比较两个集合在每个哈希函数下的最小值是否相等，并统计相等的次数。最后我们输出相等次数占总哈希函数数量的比例。

这里需要小心 C++ 的整数除法问题，我们需要将相等次数转换为浮点数进行除法运算。

```
for(int i = 0; i < q; i++){
    int x, y;
    cin >> x >> y;
    x--; y--; // 转换为0-based索引
    double similarity = 0.0;
    for(int t = 0; t < T; t++){
        if(minn[x][t] == minn[y][t]){
            similarity++;
        }
    }
    cout << double(similarity * 1.0 / T) << endl;
}
```

4. 完整代码

```

#include<bits/stdc++.h>
using namespace std;
const int T = 70; // 进行70次哈希函数

int hashFunction(int x, int t, vector<int>& k, vector<int>& mod){
    return (x ^ k[t]) * 1LL * k[t] % mod[t];
}

int main(){
    int n, m, q;
    cin >> n >> m >> q;
    //a[i]表示第i行的元素
    vector<vector<int>>> a(n, vector<int>(m));
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cin >> a[i][j];
        }
    }

    srand(time(0));
    // 生成T个随机哈希函数的参数
    vector<int> k(T), mod(T);
    for(int i = 0; i < T; i++){
        k[i] = rand() % 1000000000 + 1;
        mod[i] = rand() % 1000000000 + 1;
    }

    //计算每个集合中的最小哈希值

    vector<vector<int>>> minn(n, vector<int>(T, INT_MAX));
    for(int i = 0; i < n; i++){
        for(int t = 0; t < T; t++){
            for(int j = 0; j < m; j++){
                int hashValue = hashFunction(a[i][j], t, k, mod);
                minn[i][t] = min(minn[i][t], hashValue);
            }
        }
    }

    // 处理查询
    for(int i = 0; i < q; i++){
        int x, y;
        cin >> x >> y;
        x--; y--; // 转换为0-based索引
        double similarity = 0.0;
        for(int t = 0; t < T; t++){
            if(minn[x][t] == minn[y][t]){
                similarity++;
            }
        }
        cout << double(similarity * 1.0 / T) << endl;
    }

    return 0;
}

```